

# LABORATORIO DI INTELLIGENZA ARTIFICIALE APPLICATA

A.K.A. **AI APPLICATIONS  
IN PRACTICE**

**12 - Neural Networks and PyTorch**

Prof. Tiberio Uricchio  
< tiberio.uricchio@unipi.it >

***NEURAL NETWORKS***

# Introduction

We start this discussion by introducing Artificial Neural Network (ANN) architectures and Multilayer Perceptrons (MLPs).

History: ANNs gained popularity in 1940s and 1980s.

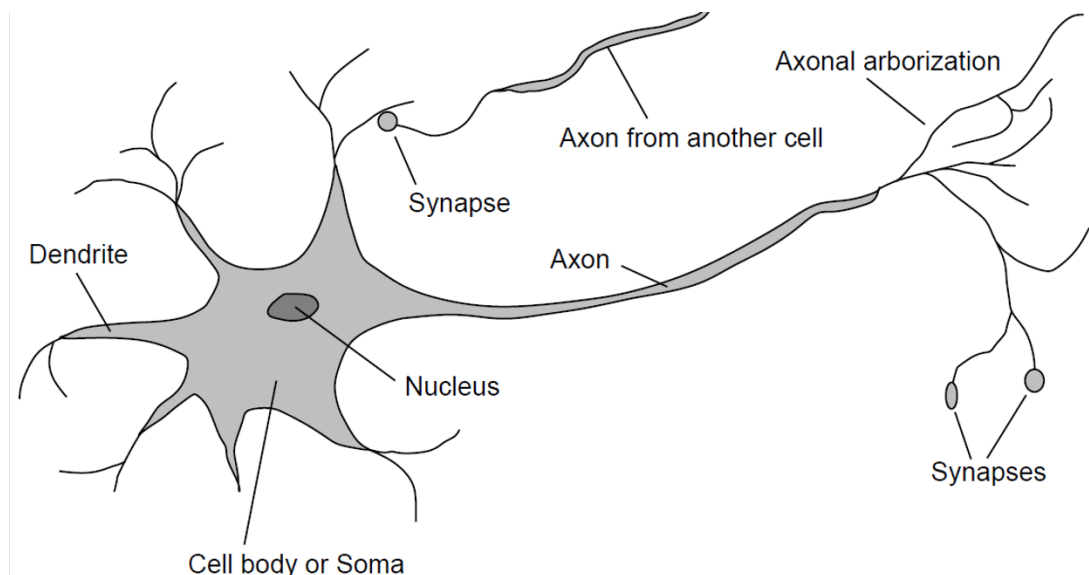
Why did deep learning become so popular again?

- Massive amounts of data
- Increase in computing power
- Better understanding of the optimization landscape

3

## Neuron

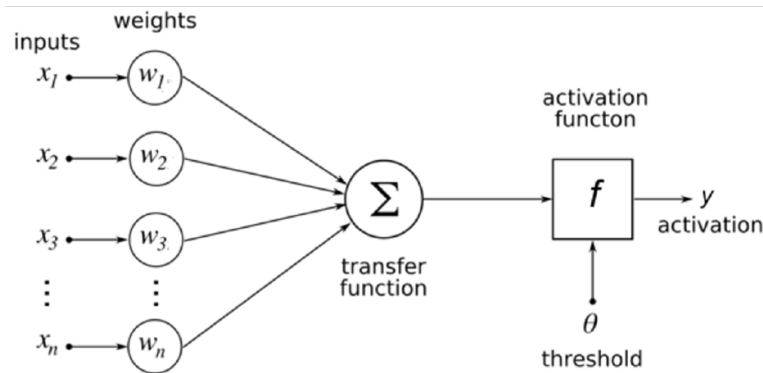
The fundamental unit of all nervous tissue, where information processing takes place.



4

# Artificial Neuron

Mathematical (simplified) model of the biological neuron



$$y = f\left(\sum_{i=1}^n w_i x_i - \theta\right)$$

The neuron performs the weighted sum of the  $n$  inputs ( $x_i$ , with  $w_i$  weights), which is the analogue of the "activation potential" of the neuron;

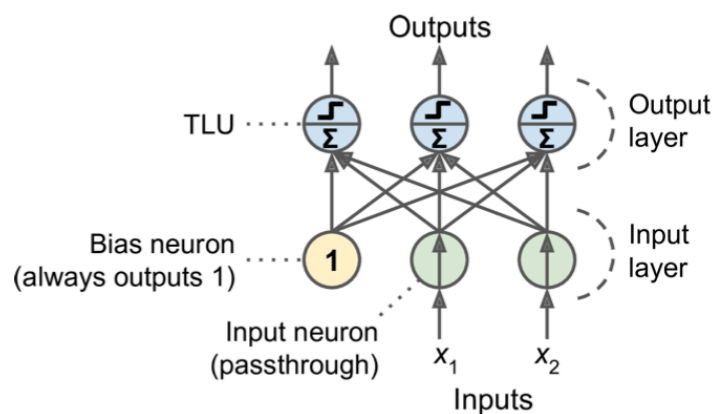
The nonlinear function  $f$  (activation function) with threshold input ( $\theta$ ) transforms the weighted sum of the inputs into the output value ( $y$ );

5

# Perceptron

A perceptron is composed of a single layer of units, with each unit connected to all the inputs (neurons in the previous layer)

Example: 2 input neurons, 1 bias neuron, and 3 output neurons



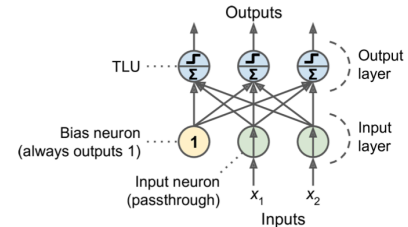
Called **fully connected or dense layer**

6

# Computing the outputs

We can find a compact representation of outputs using linear algebra

- Number of instances:  $n$  , number of features:  $d$
- Number of neurons:  $m$



$$h(\mathbf{X}) = \phi(\mathbf{XW} + \mathbf{b})$$

Where

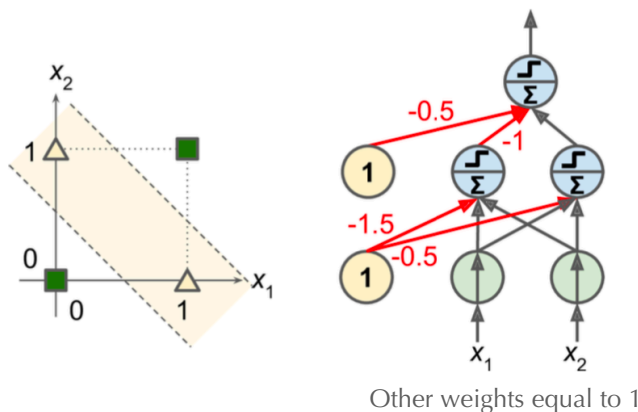
- $\mathbf{X} \in \mathbb{R}^{n \times d}$  is the Matrix of input features
- $\mathbf{W} \in \mathbb{R}^{d \times m}$  is the Matrix of connection weights
- $\mathbf{b} \in \mathbb{R}^m$  is the bias vector, one term per neuron
- $\phi$  is the activation function (Heaviside step, logistic function, ...)

7

## Multilayer Perceptrons (MLPs)

Some of the limitations of Perceptrons can be eliminated by stacking multiple Perceptrons.

For example, the following network solves the Exclusive OR (XOR) problem:

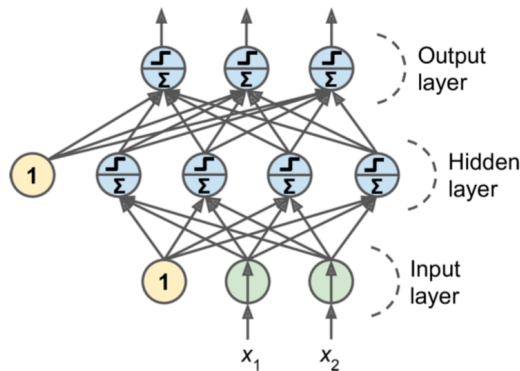


8

# Architecture of Multilayer Perceptrons

MLP is composed of one input layer, one or more layers of TLUs (called hidden layers), and one final layer of TLUs (called output layer).

Every layer except the output layer includes a bias neuron and is fully connected.



$$h(\mathbf{X}) = \phi(\phi(\mathbf{X}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2)$$

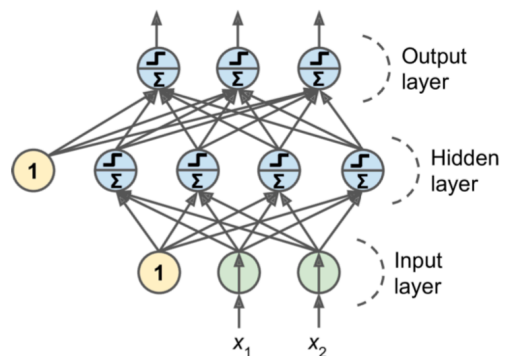
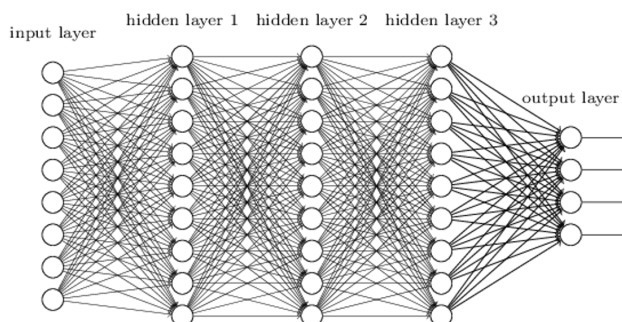
Input layer: 2 neurons, hidden layer: 4 neurons, and output layer: 3 neurons

9

## Terminology

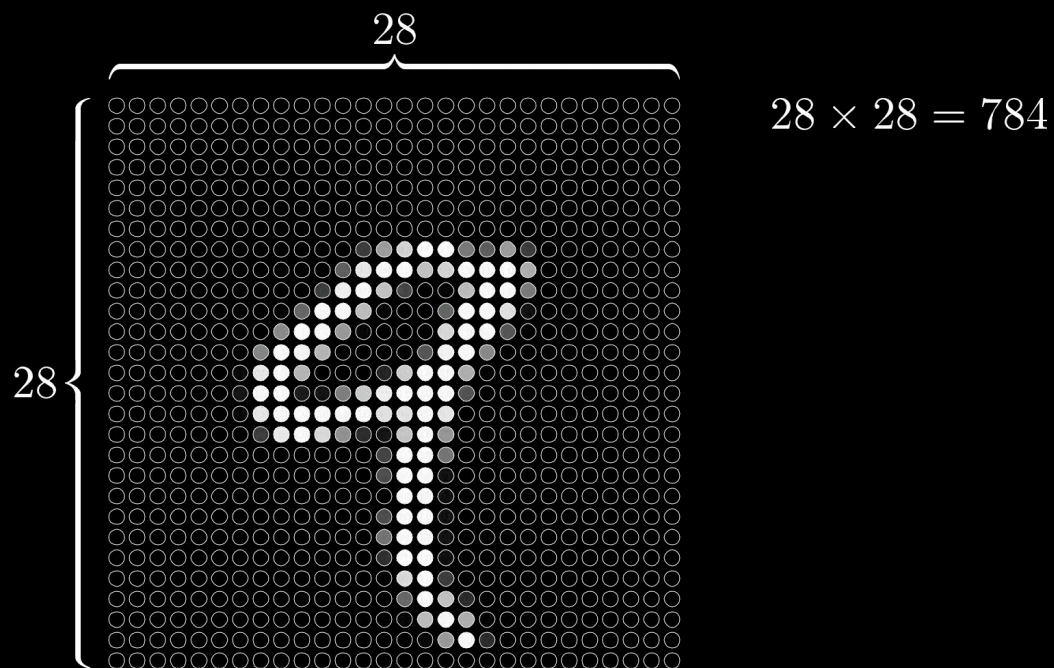
Features flow in one direction from the inputs to the outputs, so the architecture in the previous slide is an example of feedforward neural networks.

When an ANN contains a deep stack of hidden layers (e.g., tens or hundreds of layers), it is called a deep neural network (DNN) (at least 2 hidden).



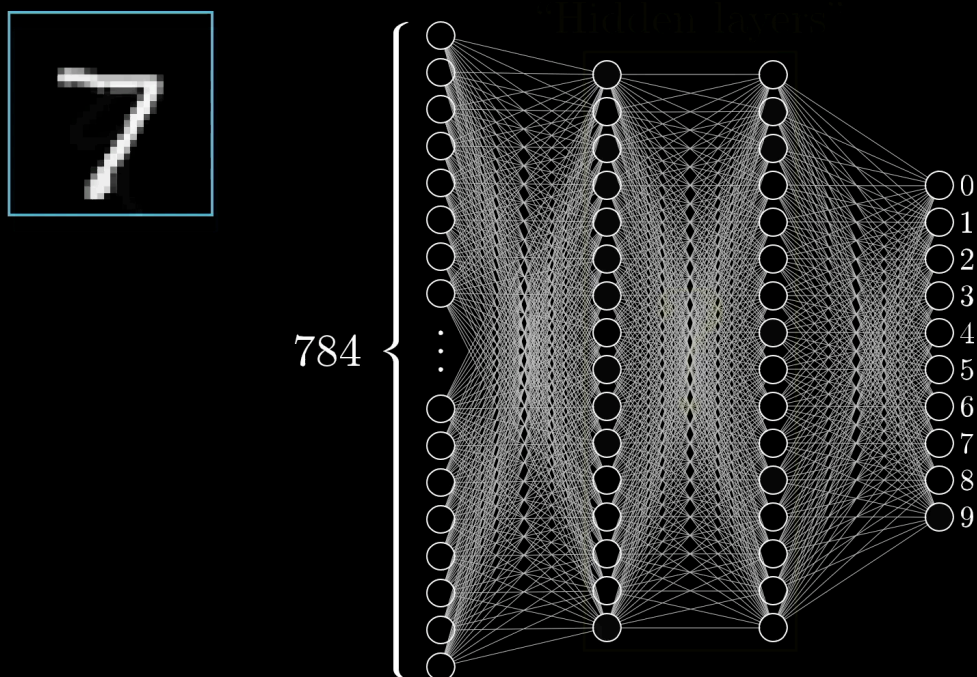
10

## Prediction for MNIST



11

## Prediction for MNIST



12

# Activation functions

The original activation function was the logistic function, due to the well-defined non-zero derivative everywhere (in contrast to the Heaviside function).

Other choices:

- Hyperbolic tangent function  $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

The output value ranges from -1 to 1

Each layer's output more or less centered around 0

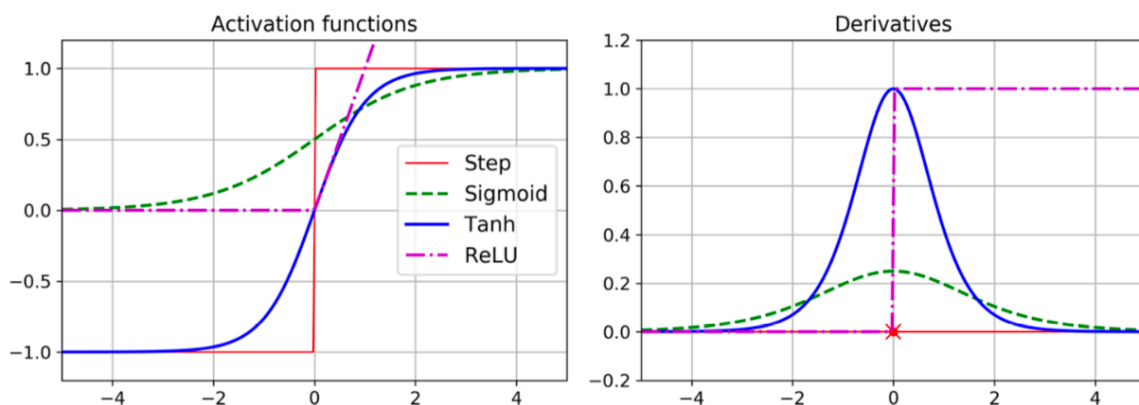
- Rectified Linear Unit function  $\text{ReLU}(x) = \max(0, x)$

Continuous but not differentiable at  $x = 0$

The default activation function because it works very well

13

# Activation functions



14

# Why activation functions?

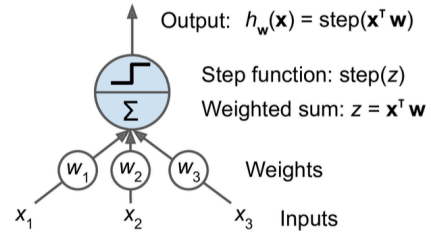
Why do we need activation functions in the first place?

- Example: consider two linear transformations

$$f(x) = 2x + 3$$

$$g(x) = 5x + 1$$

$$f(g(x)) = 2(5x-1)+3 = 10x + 1$$



If we don't have some form of non-linearity between layers, then the network is equivalent to a single layer network, incapable of solving complex problems.

On the other hand, a large enough deep neural network with non-linear activations can theoretically approximate **any continuous function**.

15

## Regression MLPs

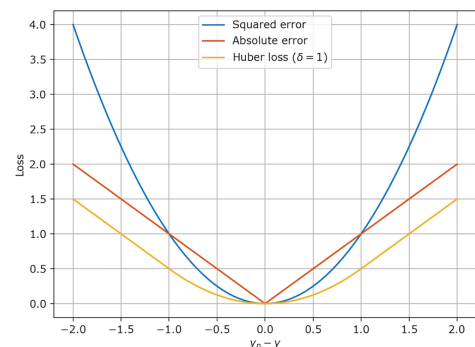
For multivariate regression (i.e., predicting multiple values), you need one output neuron per output dimension.

We typically don't use any activation function for the output neurons so they are free to produce any range of values.

- If the output is always positive, you can use ReLU
- If the output falls within a range of values, you can use logistic function and scale the labels to the range 0 - 1

Loss functions:

- Mean squared error (MSE)
- Mean absolute error (MAE)
- Huber loss





# Classification MLPs

For binary classification tasks, you just need a single output neuron with the logistic activation function.

If each instance belongs only to a single class, then we need one neuron per class with the softmax activation function.

- Example, classes 0 through 9 for digit image classification

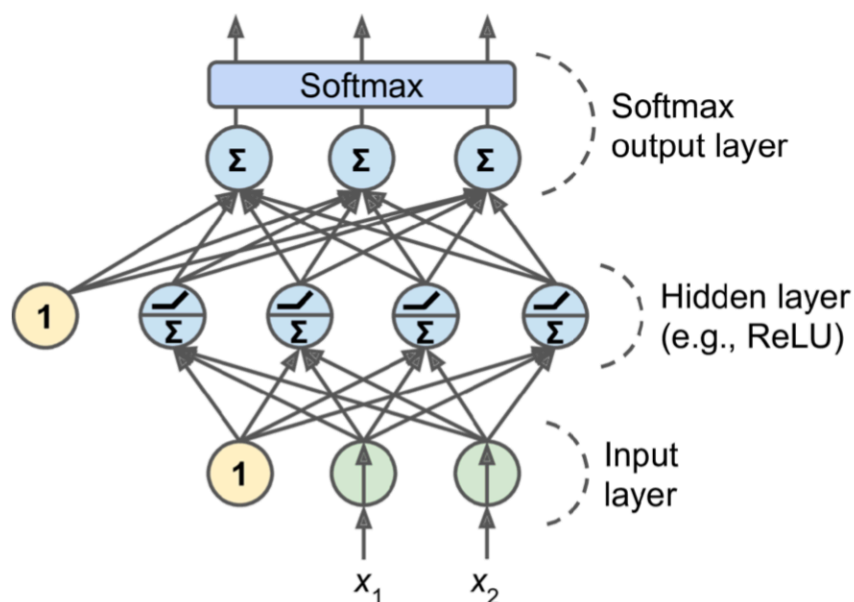
Loss function: we use the cross-entropy loss because the output layer is predicting probability distributions

$$-\sum_{c=1}^C y_c \log(p_c)$$

Where  $y_c$ : binary indicator (0 or 1);  $p_c$ : predicted probabilities.

17

## A modern MLP architecture for classification



18

# General Learning Procedure

- We need to find out how each connection weight and bias term should be optimized in order to reduce the error
- We can use Gradient Descent and its variants if we compute the gradients
- General approach:
  - For each training instance, we first make a prediction (forward pass)
  - Measure the output error using a loss function
  - Go through each layer in reverse to measure the error contribution from each connection (reverse pass)
  - Update the connection weights to reduce the error
- We apply the chain rule in the **backpropagation algorithm**
- Note that we need to initialize connection weights randomly

19

## Backpropagation algorithm

It's the backbone of deep learning, allowing networks to learn by efficiently computing the gradient of the loss function with respect to the weights of the network. Discovered in the 1960s-1970s, backpropagation became widely used in the 1980s and remains fundamental today.

It is basically the computation of the gradient w.r.t. the weight we want to update. It is based on the chain rule from the output of the network, where the loss gives a correction signal, back to the weight in question.

Given the input features  $X$

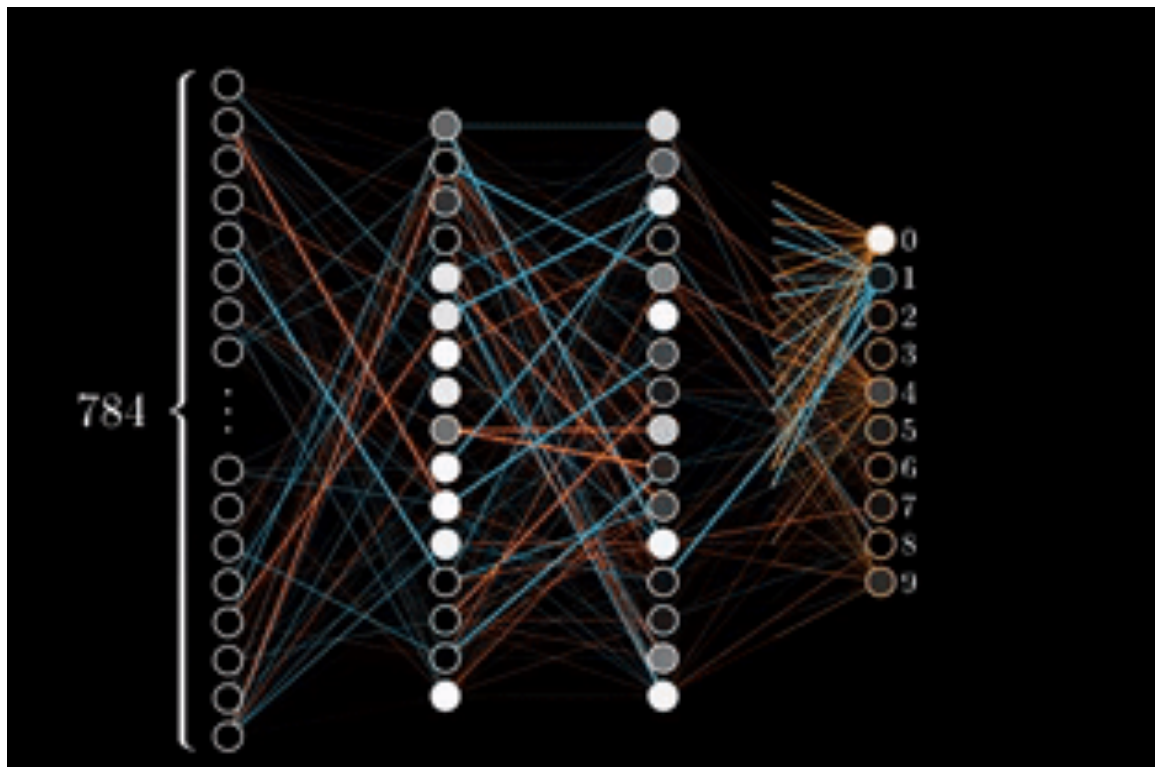
Target outputs  $Y$

Network weights  $W$

A loss function  $L(Y, \hat{Y})$  measuring prediction error

**The goal:** find weights  $W$  that minimize the loss function by adjusting them iteratively based on error gradients.

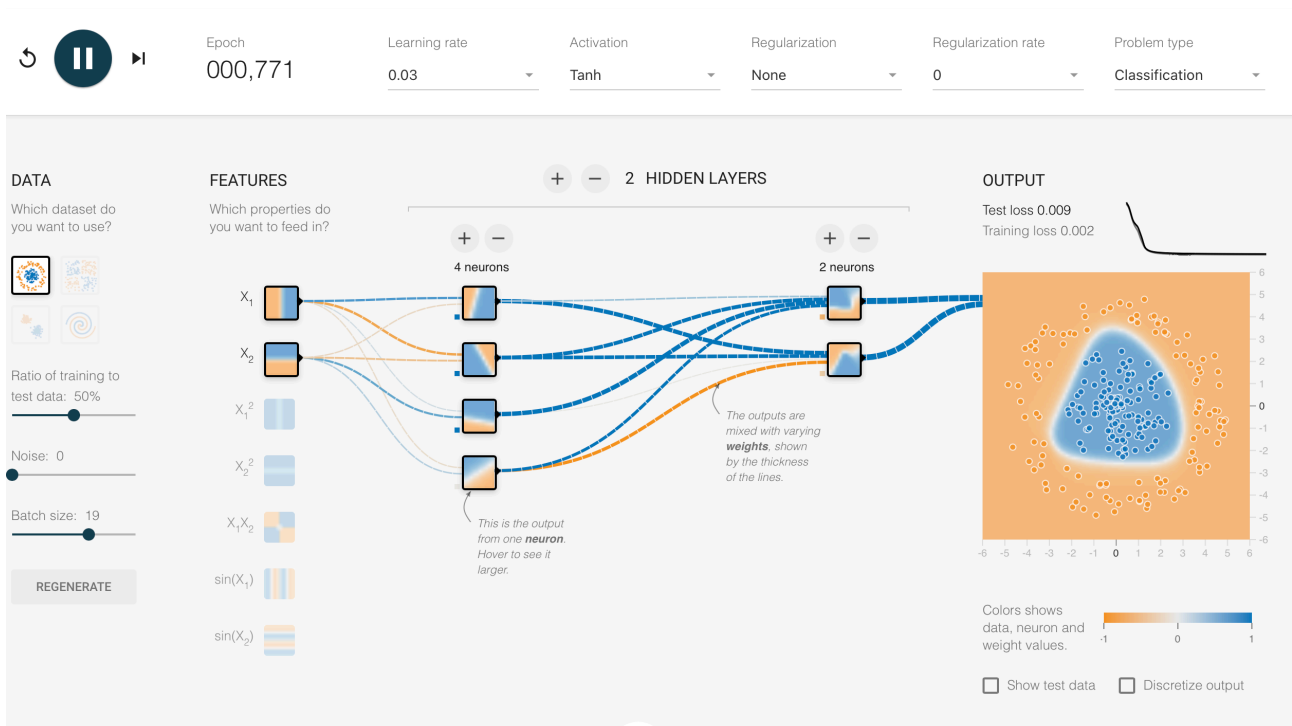
# Backpropagation



21

## Demo

<https://playground.tensorflow.org/>



22

## ***PYTORCH***

# What is PyTorch?

PyTorch is an open-source machine learning library developed by Facebook's AI Research lab (FAIR).

It provides a flexible framework for building and training neural networks with these key characteristics:

- Dynamic computational graph (define-by-run)
- Python-native feel with imperative programming style
- GPU acceleration for tensor computations
- Comprehensive ecosystem for research and production

Unlike other frameworks that use static computational graphs (define-then-run), PyTorch builds the graph on-the-fly as operations are performed, making debugging and development more intuitive.

# Tensors - The Foundation

Tensors are the fundamental data structure in PyTorch, similar to NumPy arrays but with GPU acceleration capabilities.

```
import torch

# Creating tensors
x = torch.tensor([1, 2, 3, 4])    # From Python list
y = torch.zeros(3, 4)            # All zeros tensor of shape (3,4)
z = torch.rand(2, 3)             # Random tensor with values in [0,1)
w = torch.randn(2, 3)            # Random tensor (standard normal distribution)

# Moving tensors to GPU (if available)
if torch.cuda.is_available():
    x_gpu = x.to('cuda')         # Move to GPU
    # or x_gpu = x.cuda()

# Basic tensor operations
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
c = a + b                        # Element-wise addition: [5, 7, 9]
d = torch.matmul(a, b)           # Dot product: 32
```

25

# Automatic Differentiation with autograd

PyTorch's autograd package provides automatic differentiation for all operations on tensors. This is crucial for computing gradients needed in neural network training.

```
# Create tensors with gradient tracking
x = torch.tensor([2.0], requires_grad=True)
y = torch.tensor([3.0], requires_grad=True)

# Perform operations
z = x**2 + y**3

# Compute gradients
z.backward()

# Access gradients
print(x.grad)    # dz/dx = 2*x = 4.0
print(y.grad)    # dz/dy = 3*y^2 = 27.0
```

- Dynamic computation graph built on-the-fly
- `requires_grad=True` enables gradient tracking
- `backward()` computes gradients with respect to all tensors with `requires_grad=True`

26

# Logistic Regression

PyTorch provides the `torch.nn` module with building blocks:

```
import torch.nn as nn

# Define logistic regression model
class LogisticRegressionModel(nn.Module):
    def __init__(self, input_dim):
        super(LogisticRegressionModel, self).__init__()
        # Linear layer + Sigmoid activation = Logistic Regression
        self.linear = nn.Linear(input_dim, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        # x shape: [batch_size, input_dim]
        logits = self.linear(x)
        y_pred = self.sigmoid(logits)
        return y_pred.squeeze() # Remove extra dimension

# Instantiate the model
model = LogisticRegressionModel(input_dim)
criterion = nn.BCELoss() # Binary Cross Entropy Loss
optimizer = optim.SGD(model.parameters(), lr=0.1)
```

27

# Neural Network Building Blocks

For instance for a simple neural network made of two layers:

```
import torch.nn as nn

# Simple neural network with two hidden layers
class SimpleNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(SimpleNN, self).__init__()
        self.layer1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.layer2 = nn.Linear(hidden_size, hidden_size)
        self.output = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        x = self.layer1(x)
        x = self.relu(x)
        x = self.layer2(x)
        x = self.relu(x)
        x = self.output(x)
        return x

# Instantiate the model
model = SimpleNN(input_size=10, hidden_size=50, output_size=2)
```

28

# Loss Functions and Optimizers

Training neural networks requires:

- A loss function to measure the error
- An optimizer to update the weights

```
import torch.optim as optim

# Define the model
model = SimpleNN(input_size=10, hidden_size=50, output_size=2)

# Define the loss function
criterion = nn.CrossEntropyLoss()    # For classification
# Alternative loss functions:
# criterion = nn.MSELoss()            # For regression
# criterion = nn.BCELoss()            # For binary classification

# Define the optimizer
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
# Alternative optimizers:
# optimizer = optim.Adam(model.parameters(), lr=0.001)
# optimizer = optim.RMSprop(model.parameters(), lr=0.01)
```

29

# Loss Functions and Optimizers

There are implementations for common loss and optimization algorithms:

Common loss functions:

- **nn.MSELoss()**: Mean Squared Error for regression
- **nn.CrossEntropyLoss()**: For multi-class classification
- **nn.BCELoss()**: Binary Cross Entropy for binary classification

Common optimizers:

- **optim.SGD**: Stochastic Gradient Descent (with or without momentum)
- **optim.Adam**: Adaptive learning rate optimization
- **optim.RMSprop**: Root Mean Square Propagation

30

# Training loop

Since we are using gradient based optimization, we usually define a training loop in PyTorch following these steps:

```
# Assume we have a DataLoader providing batches
for epoch in range(num_epochs):
    for batch_idx, (data, target) in enumerate(train_loader):
        # 1. Reset gradients
        optimizer.zero_grad()

        # 2. Forward pass
        output = model(data)

        # 3. Compute loss
        loss = criterion(output, target)

        # 4. Compute gradients
        loss.backward()

        # 5. Update weights
        optimizer.step()

    # Print statistics
    if batch_idx % 100 == 0:
        print(f'Epoch: {epoch}, Batch: {batch_idx}, Loss: {loss.item()}')
```

31

# Data loading with DataLoaders

```
from torch.utils.data import Dataset, DataLoader
import numpy as np

# Custom dataset
class MyDataset(Dataset):
    def __init__(self, data, targets):
        self.data = data
        self.targets = targets

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        x = self.data[idx]
        y = self.targets[idx]
        return x, y

# Create a dataset from NumPy arrays
data = np.random.randn(1000, 10)
targets = np.random.randint(0, 2, (1000,))
dataset = MyDataset(data, targets)

# Create a DataLoader
train_loader = DataLoader(
    dataset,
    batch_size=32,      # Samples per batch
    shuffle=True,       # Shuffle data
    num_workers=4       # Parallel data loading processes
)
```

Training samples can be loaded directly in memory or in GPU and handled by hand, but it is inefficient since it runs in the main thread.

DataLoaders are specialised classes to efficiently load and preprocess data.

32



# Model evaluation

Model evaluation should be done during and after training, to check that effectively we are going in the right direction. This is how it is done:

```
# Set model to evaluation mode
model.eval()

# Disable gradient computation for evaluation
with torch.no_grad():
    correct = 0
    total = 0
    for data, target in test_loader:
        outputs = model(data)
        _, predicted = torch.max(outputs.data, 1)
        total += target.size(0)
        correct += (predicted == target).sum().item()

    accuracy = 100 * correct / total
    print(f'Accuracy: {accuracy}%')

# Set model back to training mode
model.train()
```

33

# Saving and Loading Models

PyTorch allows you to save and load models for later use. It is best to save checkpoints during training, if we want to resume the learning later.

Since the optimiser has its own parameters, also remember to save them!

```
# Saving a model (two methods)
# 1. Save the entire model
torch.save(model, 'complete_model.pth')

# 2. Save only the model parameters (recommended)
torch.save(model.state_dict(), 'model_params.pth')

# Loading a model
# 1. Load the entire model
loaded_model = torch.load('complete_model.pth')

# 2. Load the parameters into a model instance
model = SimpleNN(input_size=10, hidden_size=50, output_size=2)
model.load_state_dict(torch.load('model_params.pth'))
model.eval() # Set to evaluation mode
```

34

# Transfer Learning with PyTorch

Transfer learning leverages pre-trained models to solve new tasks with limited data. PyTorch include already trained models that we can exploit.

It is useful when we have limited data.

```
import torchvision.models as models

# Load a pre-trained ResNet model
resnet = models.resnet18(pretrained=True)

# Freeze all layers
for param in resnet.parameters():
    param.requires_grad = False

# Replace the final fully connected layer
num_features = resnet.fc.in_features
resnet.fc = nn.Linear(num_features, new_num_classes)

# Now only the parameters of the new layer will be updated during training
```

35

# GPU Acceleration

One of PyTorch's key strengths is seamless GPU acceleration. CUDA operations are automatically carried out, and it is easy to switch between GPU and CPU.

```
# Check if GPU is available
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {device}')

# Create a tensor on the appropriate device
x = torch.rand(100, 100, device=device)

# Move model to GPU
model = SimpleNN(10, 50, 2).to(device)

# During training, move data to device
for data, target in train_loader:
    data, target = data.to(device), target.to(device)
    output = model(data)
    # ...
```

36

# Debugging and visualization

Beyond manual debug and visualization, PyTorch include tools for showing the training process with its metrics. Tensorboard is a web interface for that.

```
# Print model architecture
print(model)

# Summary of model parameters
total_params = sum(p.numel() for p in model.parameters())
print(f'Total parameters: {total_params}')

# Visualize training with TensorBoard
from torch.utils.tensorboard import SummaryWriter

writer = SummaryWriter('runs/experiment_1')

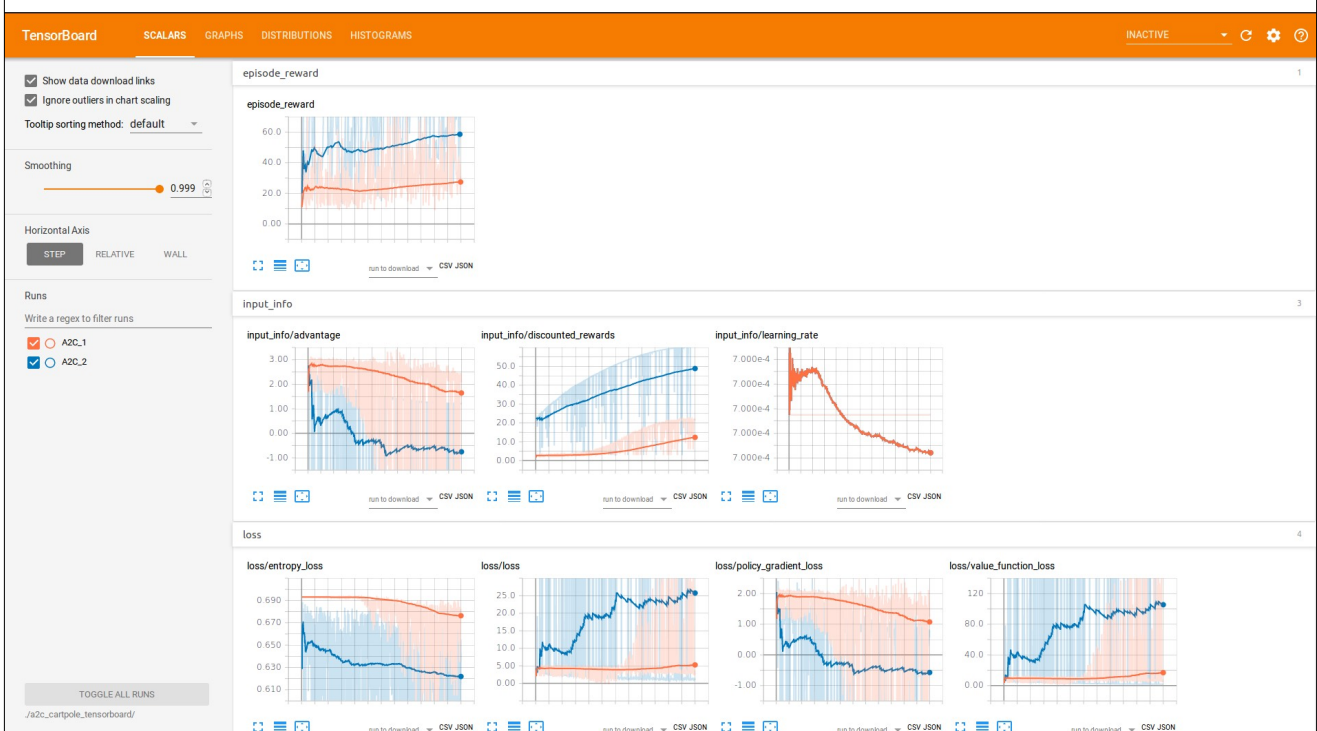
# Log scalars (e.g., loss) during training
for epoch in range(num_epochs):
    # training code...
    writer.add_scalar('Loss/train', running_loss, epoch)
    writer.add_scalar('Accuracy/train', train_accuracy, epoch)

# Visualize model graph
dummy_input = torch.zeros(1, 10)
writer.add_graph(model, dummy_input)

writer.close()
```

37

# Debugging and visualization



38

# PyTorch Ecosystem

PyTorch has a rich ecosystem of libraries built around it:

- **TorchVision:** for computer vision tasks
  - Pre-trained models (ResNet, VGG, etc.)
  - Datasets (MNIST, CIFAR, ImageNet)
  - Transformations and utilities
- **TorchText:** For NLP tasks
  - Text processing utilities
  - Word embeddings
  - Language datasets
- **TorchAudio:** For audio processing
  - Audio datasets
  - Signal processing functions
  - Audio transformations

39

# From Training to Deployment

PyTorch models can be deployed to production using:

- TorchScript: for serializing and optimizing models

```
scripted_model = torch.jit.script(model)
scripted_model.save('scripted_model.pt')
```

- ONNX: Open Neural Network Exchange format

```
dummy_input = torch.randn(1, 10)
torch.onnx.export(model, dummy_input, 'model.onnx')
```

- TorchServe: for serving PyTorch models
- PyTorch Mobile: For on-device deployment

40

# Practical Tips and Best Practices

Recommendations for effective PyTorch development:

- **Start small** - begin with simple architectures and gradually add complexity
- **Monitor gradients** - watch for exploding/vanishing gradients during training
- **Normalize inputs** for faster convergence
- Check CUDA **memory usage**

```
torch.cuda.memory_allocated(), torch.cuda.memory_reserved()
```

- **Gradient clipping** for stabilizing training

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

41

## Exercise

### Classification on MNIST Dataset

- Build a classifier for the MNIST dataset (handwritten digits)

42

# Solution

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# 1. Data preparation
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

train_dataset = datasets.MNIST('data', train=True, download=True, transform=transform)
test_dataset = datasets.MNIST('data', train=False, transform=transform)

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=1000)

# 2. Define the model
class MNISTClassifier(nn.Module):
    def __init__(self):
        super(MNISTClassifier, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, 1)
        self.conv2 = nn.Conv2d(32, 64, 3, 1)
        self.dropout1 = nn.Dropout2d(0.25)
        self.dropout2 = nn.Dropout2d(0.5)
        self.fc1 = nn.Linear(9216, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.conv1(x)
        x = nn.functional.relu(x)
        x = self.conv2(x)
        x = nn.functional.relu(x)
        x = nn.functional.max_pool2d(x, 2)
        x = self.dropout1(x)
        x = torch.flatten(x, 1)
        x = self.fc1(x)
        x = nn.functional.relu(x)
        x = self.dropout2(x)
        x = self.fc2(x)
        return nn.functional.log_softmax(x, dim=1)

# Full training code would follow...
```